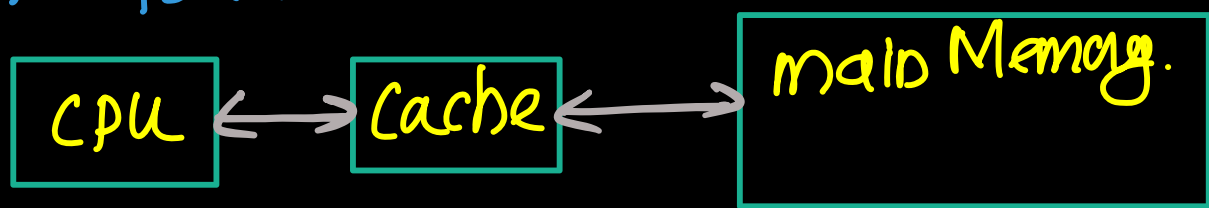


Cache Memory (Note)

- Cache memory is fast memory placed close to the processor
- it holds the commonly used data from the main memory so that processor can access it quickly.
- Cache can store both instruction and data. (most frequently used and recently used).

— how data found in cache



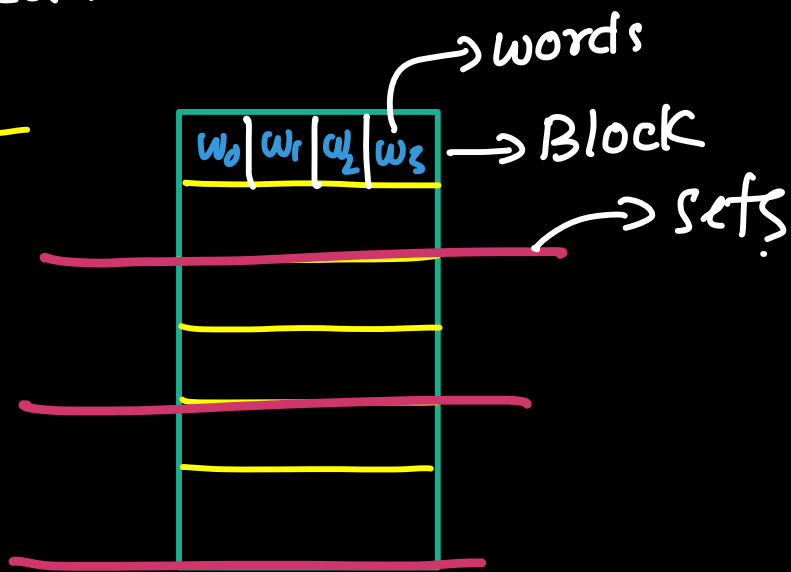
When a processor needs an instruction/data

- The CPU first checks the cache
- If the required data is present in the cache (cache hit), then processor accesses data.
- If the required data is not present in the cache, then CPU fetches from the main memory and copies to the cache for future use.
- If processor accesses the data (information) recently, it is likely to access it again soon. So fetched data is copied to the cache. So the future access will likely hit.
- If processor accesses the data from a memory location, it is likely to access nearby location soon. When cache fetches one word from main memory, it may also fetch several adjacent words.
- The group of words are called Blocks.
- no. of words in the block is called Block size

Cache organisation

— A Cache is organised into several sets. Each set can hold one or more Blocks of data

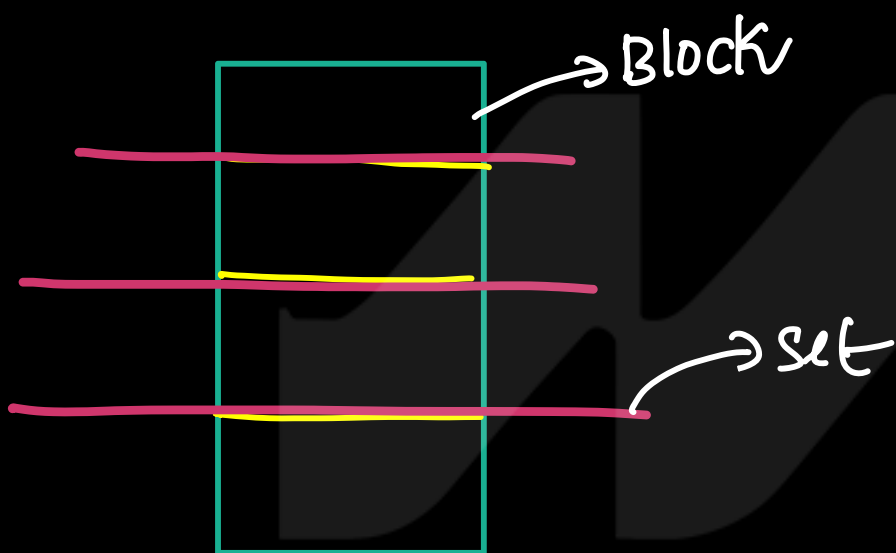
eg: 1



→ Here one sets consists of 2 Block.

no: of Block in a set = 2

eg: 2 One Block in set



eg: Cache size = 16 Byte = 16 word

1 Byte = 1 word

no: of word in Block (Block size) = 4 Byte = 4 word

$$\text{No: of Blocks} = \frac{\text{Cache size}}{\text{Block size}} = \frac{16 \text{ word}}{4 \text{ word}} = \underline{\underline{4}}$$

no: of blocks in set = 1

No: of set = 4

no: of bits to represent set = $\log_2 \text{no: of set}$

$$= \log_2 4 = \underline{\underline{2 \text{ bit}}}$$

no: of bits to represent words inside the Block = $\log_2 (\text{no: of words in a Block})$

$$= \log_2 4 = \underline{\underline{2 \text{ bit}}}$$

Cache terminology.

(N)

① Capacity (C)

— no: of data bytes in the cache

→ eg: $C = 64 \text{ Byte}$.

eg: $1 \text{ Byte} \rightarrow 1 \text{ word}$

$C = 64 \text{ word}$

— Block size (b)

Block $\rightarrow$ group of words

Block size — no: of words in a Block

eg: $b = 4 \text{ word}$

it contains 4 words

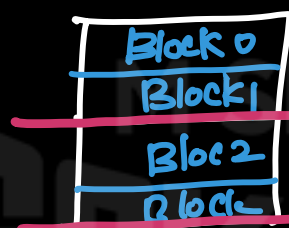
→ no: of Block (B)

$$B = \frac{\text{Capacity (C)}}{\text{no: of block (b)}} = \frac{C}{b}$$

→ Degree of associative (N)

no: of Blocks in a set

eg:



— Set No 0

— Set No-1

$N = 2$ (1 set contains 2 Block)

→ No: of sets (S)

$$S = \frac{\text{no: of Block}}{\text{no: of Block in a set}} = \frac{B}{N}$$

$$S = B/N$$

eg: Main Memory size = 4GB

Cache size = 1MB

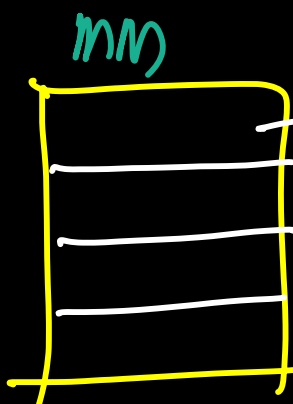
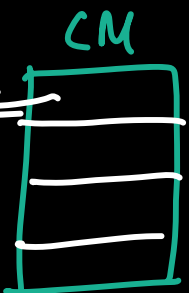
Block size = 4KB.

find physical Address

2 way set-cache

Steps. →

Cache Mem
Block



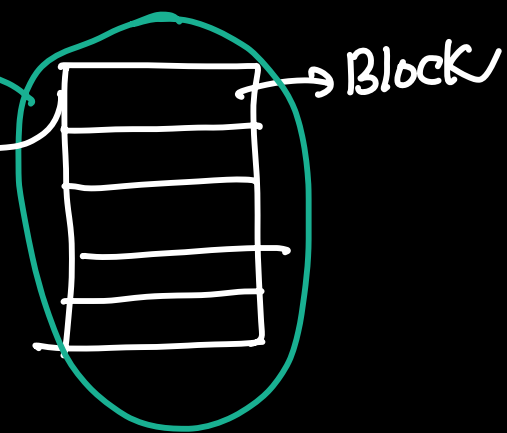
→ Main Memory Block

(A)

① Find the No: of Block in cache (B)

$$B = \frac{\text{Cache size}}{\text{block size}}$$

$$B = \frac{C}{b}$$



$$C = 1MB = 1 \times 2^{20} B$$

$$1B \rightarrow 1 \text{ word}$$

$$C = 1 \times 2^{20} \text{ word}$$

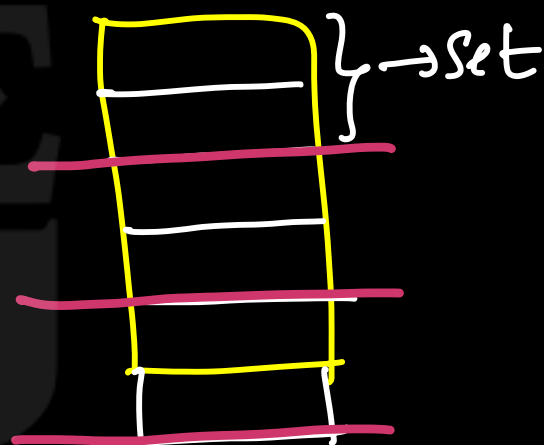
$$b = 4KB = 4 \times 2^{10} \times \text{word} = 2^{12} \text{ word}$$

$$B = \frac{2^{20}}{2^{12}} = \underline{\underline{2^8}}$$

$$\text{no: of bits to Rep } B = \log_2 2^8 = \underline{\underline{8}} \text{ bits}$$

2-way Set $\rightarrow$ 2 Blocks in 1 set

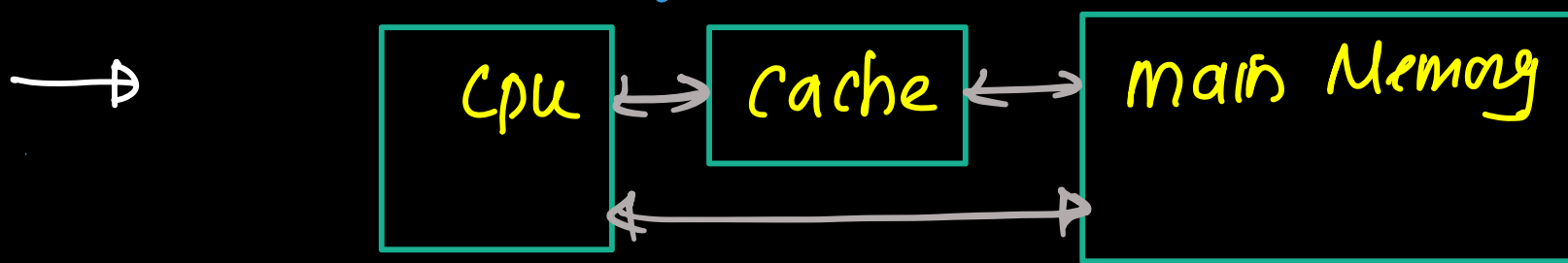
$$\text{no: of set } (S) = \frac{\text{no: of Block } (B)}{\text{no: of Block in a set } (N)}$$



$$S = \frac{2^8}{2} = \underline{\underline{2^4}}$$

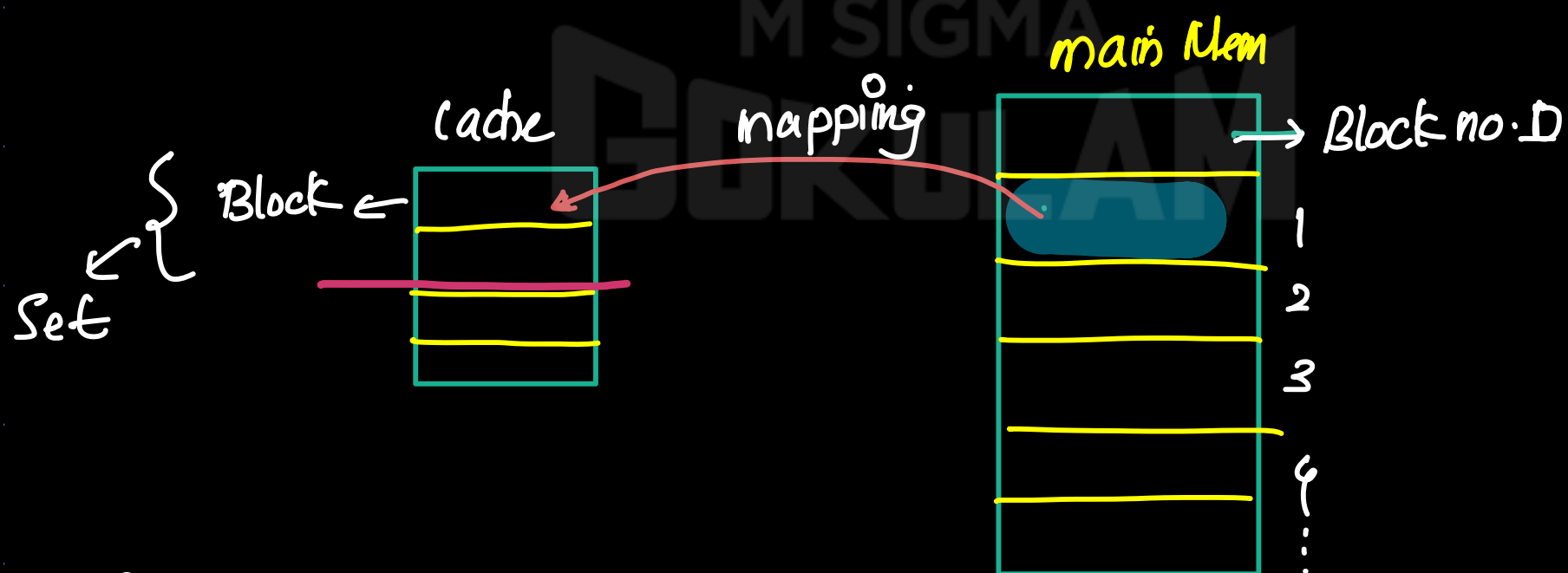
$$\text{no: of bits to represent set} = \log_2 2^4 = \underline{\underline{4}}$$

Cache Mapping technique. (Note)



— When CPU access information (data / Instruction) from the main memory, a copy of that information is placed in the cache memory. assuming that same data will be used again in the near future. Not only the requested information, but also the information stored in the neighbouring memory location is brought into the cache. This is because of the principle of locality of Reference.

— Main memory is divided into Blocks and cache memory divided into sets. each block of main memory must be mapped to the same location of cache memory.



→ Size of cache memory is limited when compared to main memory. So all memory blocks cannot be stored in the cache memory at same time. , a technique is required to decide how blocks of main memory are place into cache memory and how CPU locates the required data in the cache. This technique is called Cache mapping technique.

Set No	Cache Memory (16 bytes)			
0				
1				
2				
3				

Block No	Main Memory (64 bytes)			
0	W 0	W 1	W 2	W 3
1	W 4	W 5	W 6	W 7
2	W 8	W 9	W 10	W 11
3	W 12	W 13	W 14	W 15
4	W 16	W 17	W 18	W 19
5	W 20	W 21	W 22	W 23
6	W 24	W 25	W 26	W 27
7	W 28	W 29	W 30	W 31
8	W 32	W 33	W 34	W 35
9	W 36	W 37	W 38	W 39
10	W 40	W 41	W 42	W 43
11	W 44	W 45	W 46	W 47
12	W 48	W 49	W 50	W 51
13	W 52	W 53	W 54	W 55
14	W 56	W 57	W 58	W 59
15	W 60	W 61	W 62	W 63

Consider a cache with 4 set (one block in one set).

Block no: 0 of MM is mapped to set no 0

$$\begin{aligned} B_0 &\longrightarrow S_0 \\ B_1 &\longrightarrow S_1 \\ B_2 &\longrightarrow S_2 \\ B_3 &\longrightarrow S_3 \end{aligned}$$

Where we place remaining blocks . so we need Rule to show how we can place main memory Blocks in cache

Definition of Cache Mapping

Cache mapping is the technique used to determine how blocks of main memory are placed into cache memory and how the CPU locates the required data in the cache.

Need for Cache Mapping

- Main memory is large but slow

Accessing data directly from main memory increases CPU waiting time.

- Cache memory is small but very fast

Frequently used data can be accessed quickly if stored in cache.

- Limited cache size

Since cache cannot hold all memory blocks, a proper mapping method is required.

- Improved system performance

Cache mapping reduces average memory access time and improves overall CPU performance.

Types of Cache Mapping

Note

- ① Direct mapping (one block in a set)
- ② Fully associative mapping (only one set & all blocks are in a single set)
- ③ Set-associative mapping (2 or more blocks in a set)

Direct mapping

Cache memory is organised into a large number of sets. In direct mapping cache, each set contains only one block. It is a cache mapping technique in which each block of main memory can be stored in exactly one specific set in the cache.

Example Explanation

Consider the following:

- Main memory size = 64 bytes
- Block size = 4 bytes

$$\text{Number of blocks in main memory} = \frac{64}{4} = 16 \text{ blocks}$$

- Cache size = 16 bytes
- Cache block size = 4 bytes (same as main memory block size)

$$\text{Total blocks in cache} = \frac{16}{4} = 4 \text{ blocks}$$

Cache Organisation

- Since this is direct mapping, one set consists of one block
- Therefore:

$$\text{Number of sets in cache} = \text{Number of cache blocks} = 4$$

- Cache sets are:
 - Set 0
 - Set 1
 - Set 2
 - Set 3

Set No	Cache Memory (16 bytes)			
0				
1				
2				
3				

Block No	Main Memory (64 bytes)			
0	W 0	W 1	W 2	W 3
1	W 4	W 5	W 6	W 7
2	W 8	W 9	W 10	W 11
3	W 12	W 13	W 14	W 15
4	W 16	W 17	W 18	W 19
5	W 20	W 21	W 22	W 23
6	W 24	W 25	W 26	W 27
7	W 28	W 29	W 30	W 31
8	W 32	W 33	W 34	W 35
9	W 36	W 37	W 38	W 39
10	W 40	W 41	W 42	W 43
11	W 44	W 45	W 46	W 47
12	W 48	W 49	W 50	W 51
13	W 52	W 53	W 54	W 55
14	W 56	W 57	W 58	W 59
15	W 60	W 61	W 62	W 63

→ Block no 0 (b_0) in the main memory mapped to set no '0' (s_0) in the cache memory.

$s_0 \xleftarrow{\text{mapped}} b_0$
 $s_1 \xleftarrow{\quad} b_1$
 $s_2 \xleftarrow{\quad} b_2$
 $s_3 \xleftarrow{\quad} b_3$

Set No	Cache Memory (16 bytes)			
0				
1				
2				
3				

Block No	Main Memory (64 bytes)			
0	W 0	W 1	W 2	W 3
1	W 4	W 5	W 6	W 7
2	W 8	W 9	W 10	W 11
3	W 12	W 13	W 14	W 15
4	W 16	W 17	W 18	W 19
5	W 20	W 21	W 22	W 23
6	W 24	W 25	W 26	W 27
7	W 28	W 29	W 30	W 31
8	W 32	W 33	W 34	W 35
9	W 36	W 37	W 38	W 39
10	W 40	W 41	W 42	W 43
11	W 44	W 45	W 46	W 47
12	W 48	W 49	W 50	W 51
13	W 52	W 53	W 54	W 55
14	W 56	W 57	W 58	W 59
15	W 60	W 61	W 62	W 63

→ Now block no 4 is mapped to set no '0'

$s_0 \xleftarrow{\text{mapped}} b_4$
 $s_1 \xleftarrow{\quad} b_5$
 $s_2 \xleftarrow{\quad} b_6$
 $\vdots$

→ Mapping function

$$\text{Set no} = (\text{Block no}) \bmod (\text{no. of sets in the cache})$$

For Block no 0 → $0 \% 4 = 0 \rightarrow \text{Set no} = 0$

$b_0 \rightarrow 0 \% 4 = 0 \rightarrow s_0$
 $b_1 \rightarrow 1 \% 4 = 1 \rightarrow s_1$
 $b_2 \rightarrow 2 \% 4 = 2 \rightarrow s_2$
 $b_3 \rightarrow 3 \% 4 = 3 \rightarrow s_3$
 $b_4 \rightarrow 4 \% 4 = 0 \rightarrow s_0$
 $b_5 \rightarrow 5 \% 4 = 1 \rightarrow s_1$
 $b_6 \rightarrow 6 \% 4 = 2 \rightarrow s_2$
 $b_7 \rightarrow 7 \% 4 = 3 \rightarrow s_3$
 $b_8 \rightarrow 8 \% 4 = 0 \rightarrow s_0$

$\vdots$

$b_{11} \rightarrow 11 \% 4 = 3 \rightarrow s_3$

$b_{15} \longrightarrow 15 \div 4 = 3$

(2)

Set no	Block.
S_0	b_0, b_4, b_8, b_{12}
S_1	b_1, b_5, b_9, b_{13}
S_2	b_2, b_6, b_{10}, b_{14}
S_3	b_3, b_7, b_{11}, b_{15}



Address format in direct mapping

A physical Address is divided into 3 field

(N)



- ① Block offset field
- ② Set bit field
- ③ tag bits.

- Set bit field

these bits are used to identify which set no should be accessed

→ tag bit field

these bits are used to identify the which Block in the main memory is stored in the selected set

→ Block offset

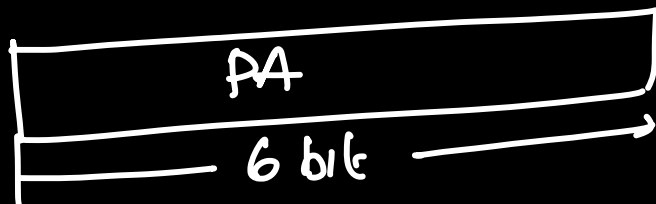
these bits are used to identify the which word inside the blocks should be needed.

MM size = 64 Byte = 64 word

1 Byte = 1 word
Block size = 4 word, cache size = 16 word

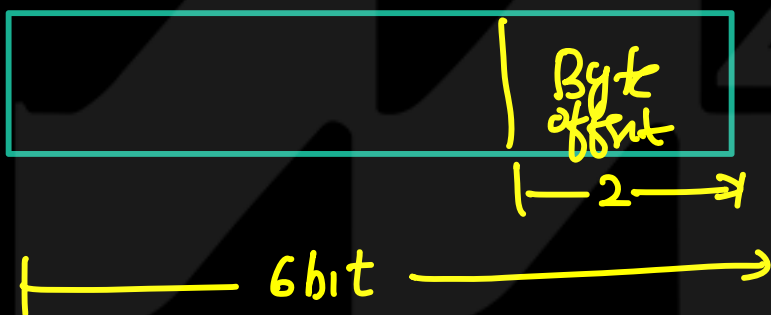
Step 1 → find physical Address bits.

MM size = 64 Byte = 64 word
no: of bits to represent MM = $\log_2 64 = 6$ bits



Step 2 → find the Block offset

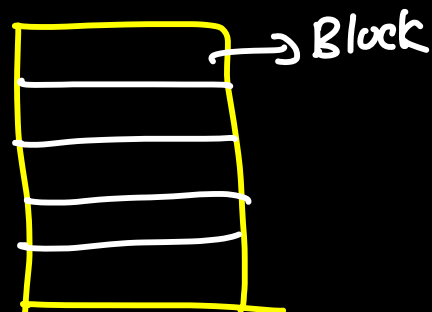
Block size = 4 word
no: of bits to represent words inside the block = $\log_2 4 = 2$ bits
Block offset = 2 bits



Step 3 — Find Set bit

Find the no: of sets in the cache.

$$\text{no: of Block in cache} = \frac{\text{Cache size}}{\text{Block size}} = \frac{16 \text{ word}}{4 \text{ word}} = \underline{\underline{4}}$$



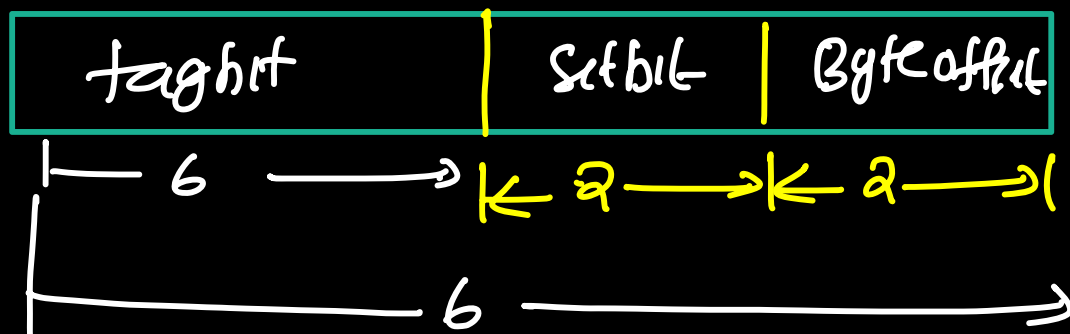
no: of block in a set = 1

Total No: of set = No: of Block = 4

Set bit = $\log_2 4 = \underline{\underline{2}}$ bit

Find tag bit

tag bit = PA bit - set bit - Byte offset



How the CPU Accesses Data from Cache (Note)

1. Processor generates a physical address (PA)

When the CPU needs data or an instruction, it generates a physical address.

This physical address is divided into three fields:

- Tag
- Set index (set bits)
- Block/offset

2. Cache organization into sets

- The cache is divided into multiple sets.
- Each set contains one blocks)
- Every Set in the cache has:
 - Tag field - identifies which block of main memory is stored
 - Valid bit - indicates whether the data in that line is valid
 - Data field - actual data copied from main memory

3. Selecting the set using set bits

- The CPU uses the set bits from the physical address to select one specific set in the cache.
- These set bits determine which set may contain the required block of main memory.

4. Tag comparison

- Within the selected set, the tag field from the physical address is compared with the tag stored in the set.
- This comparison checks whether the required memory block is present in that set.

5. Valid bit check

- At the same time, the valid bit of the selected set is checked.
- If the valid bit is 1, it means the data stored in that set is valid.

6. Cache hit condition

A cache hit occurs if:

- The tag bits match, and
- The valid bit is 1

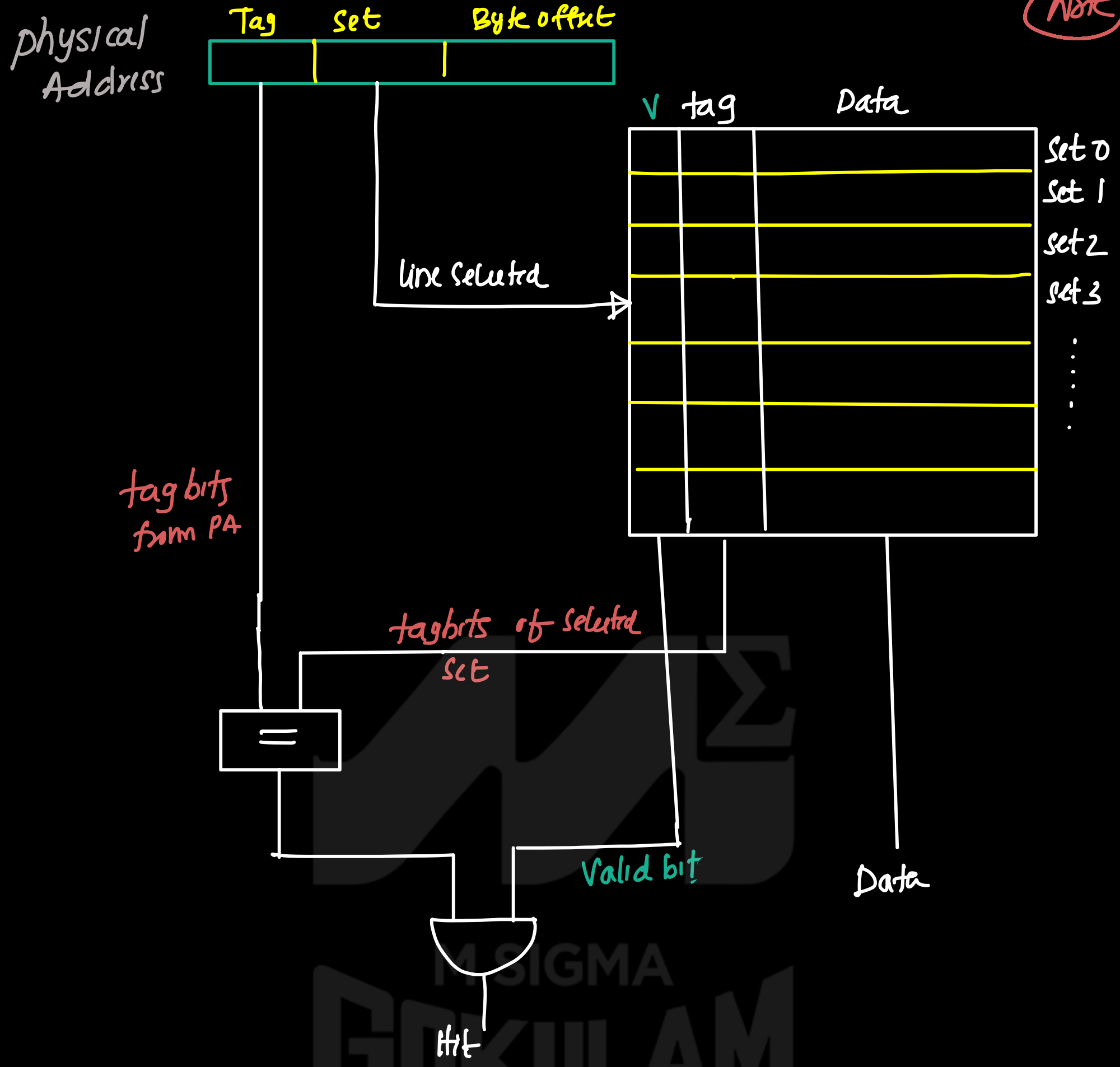
7. Data access

- When a cache hit occurs, the CPU directly copies the required data from the data field of the selected set.
- This is much faster than accessing main memory.

8. Cache miss (for completeness)

- If the tag does not match or the valid bit is 0, a cache miss occurs.
- The required block is then fetched from main memory, placed into the cache, the tag is updated, and the valid bit is set to 1.

Note



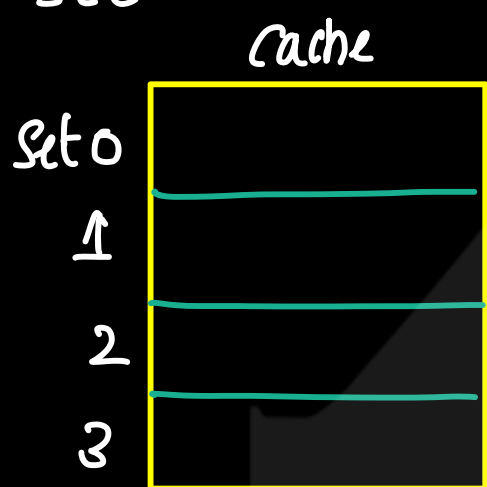
Disadvantages of Direct mapping

(N)

- In direct mapping, each memory block is mapped to exactly on one specific cache set.
- Because of this fixed mapping. Multiple main memory blocks may map to same set, which leads to block conflict

Consider a main memory with 16 block and cache memory with 4 set

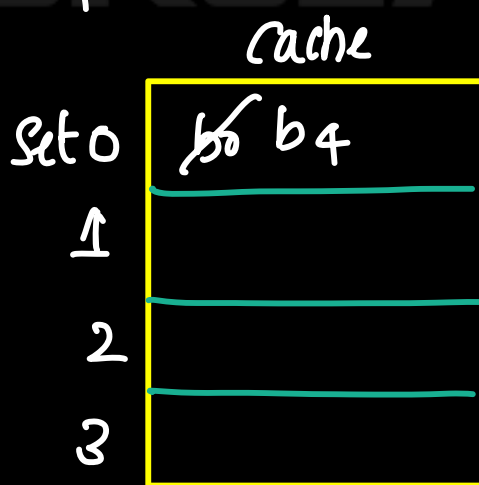
main memory block b_0, b_4, b_8, b_{12} are mapped to Set 0



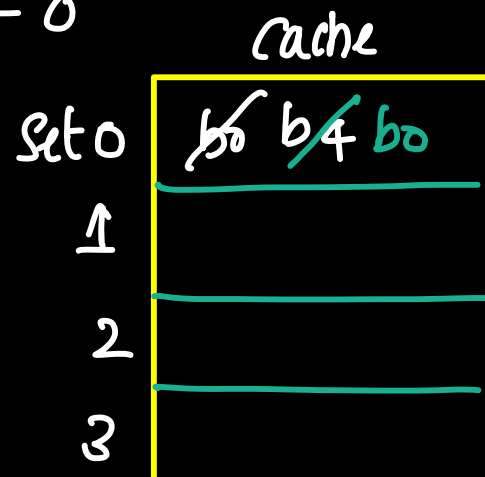
Let

cpu block request is b_0, b_4, b_0, b_4 etc.

- ① first Map the block b_0 to Set 0
- ② next block request is b_4 , so b_4 need to map to Set 0. The cache replaces block b_0 and loads b_4



- ③ 3rd Block request is b_0 , But b_0 is not available in the cache (Miss). cache replace block 4 and maps Block b_0 to Set 0



- ④ 4th request is block b_4 . but b_4 is not in the cache (Miss). cache replace block 0 and maps block 4 to set '0'.
- ⑤ Both blocks are continuously replaced in the set 0. This condition is called conflict Miss.
- ⑥ During the 2nd, 3rd and 4th block request blocks are not available in the cache. because all the requested blocks are mapped to same set.

— Conflict miss can be reduced by using other mapping techniques such as set associative mapping / fully associative mapping.

